7-2 Project Two README File

Jas King

CS-340 Client/Server Development

Prof. Helen Gebre-Amlak

February 25, 2026

1. Required Functionality of the Project

This project aims to create a client-facing dashboard application for Grazioso Salvare, a global organization focused on training rescue animals. The dashboard allows users to engage with data from animal shelters stored in a MongoDB database and find dogs that are ideal candidates for various types of rescue training.

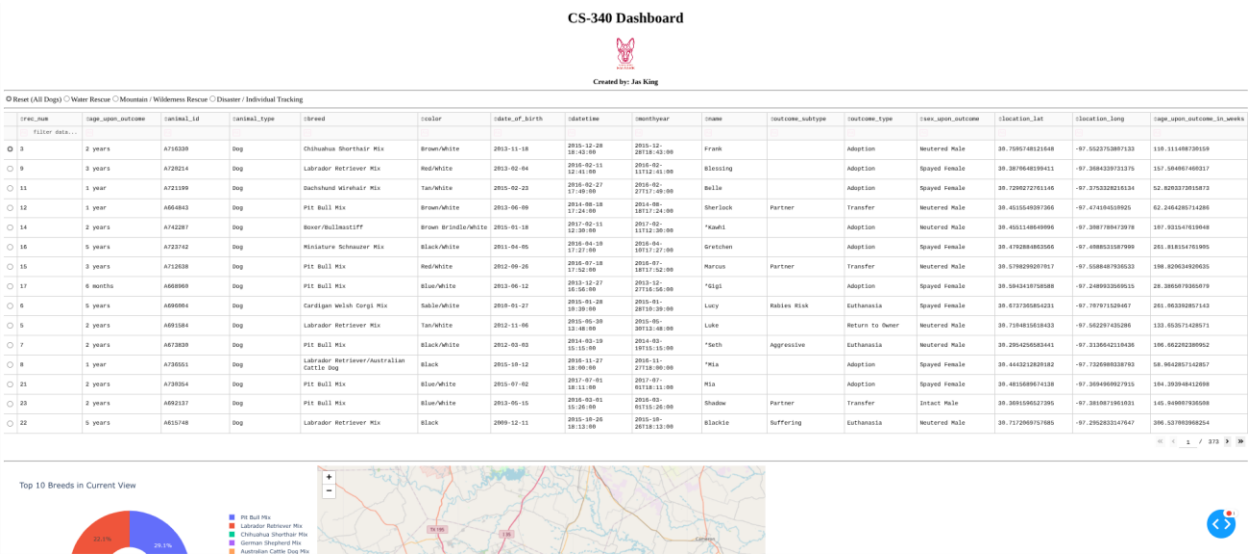
The dashboard includes the following essential features:

- Shows the Grazioso Salvare logo with a clickable link to the organization's website.
- Displays a unique developer identifier (Created by: Jas King).
- Offers interactive filter controls for users to view dogs suitable for:
    - Water Rescue
    - Mountain or Wilderness Rescue
    - Disaster or Individual Tracking
    - Reset (unfiltered dataset)
- Features an interactive data table that updates in real-time based on the selected filter.
- Includes a geolocation map that refreshes according to the filtered dataset and desired animal.
- Presents a secondary visualization (breed distribution chart) that updates dynamically with the selected filter.

The following screenshots illustrate successful operation of all required dashboard states and confirm that the necessary functionality has been achieved.

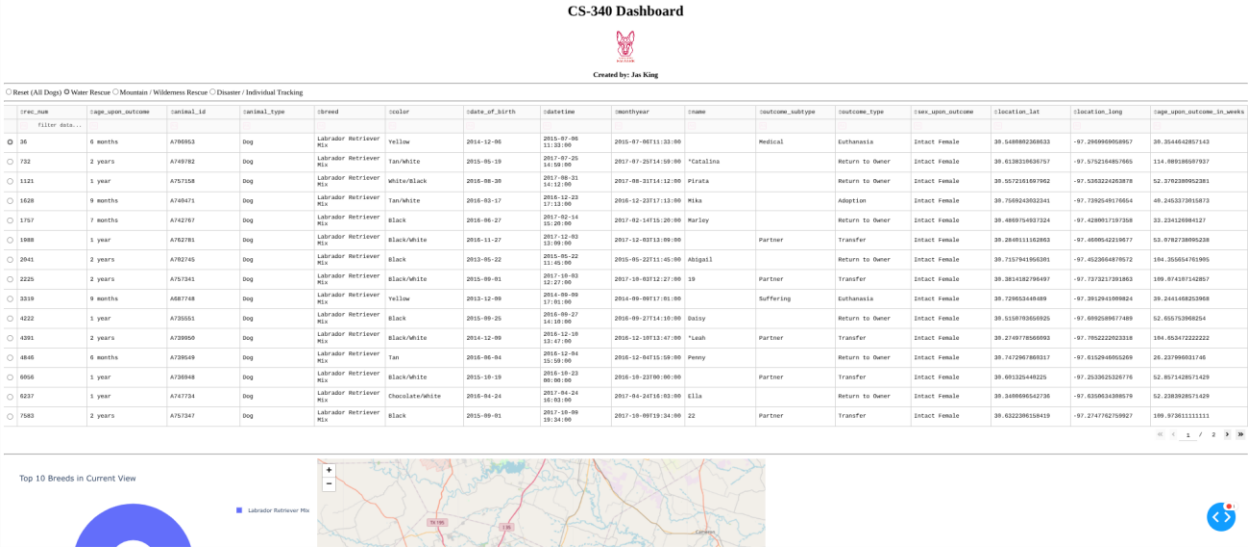
1. **Screenshots**

Initial Dashboard (Unfiltered View)



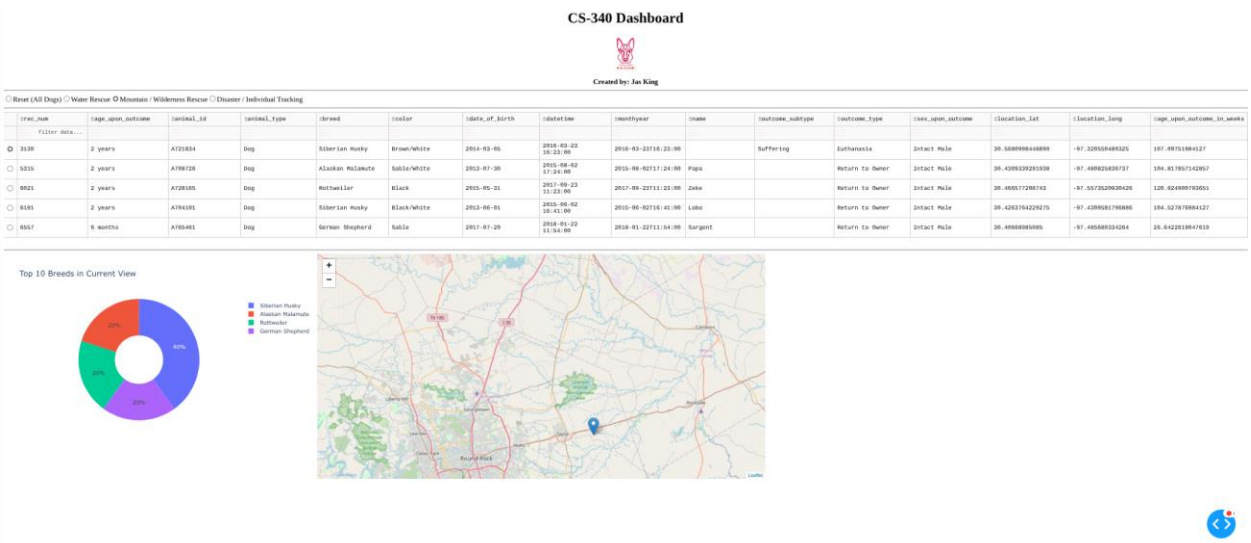
This screenshot shows the dashboard immediately after loading, with all dogs displayed and all widgets visible.

Water Rescue Filter Applied



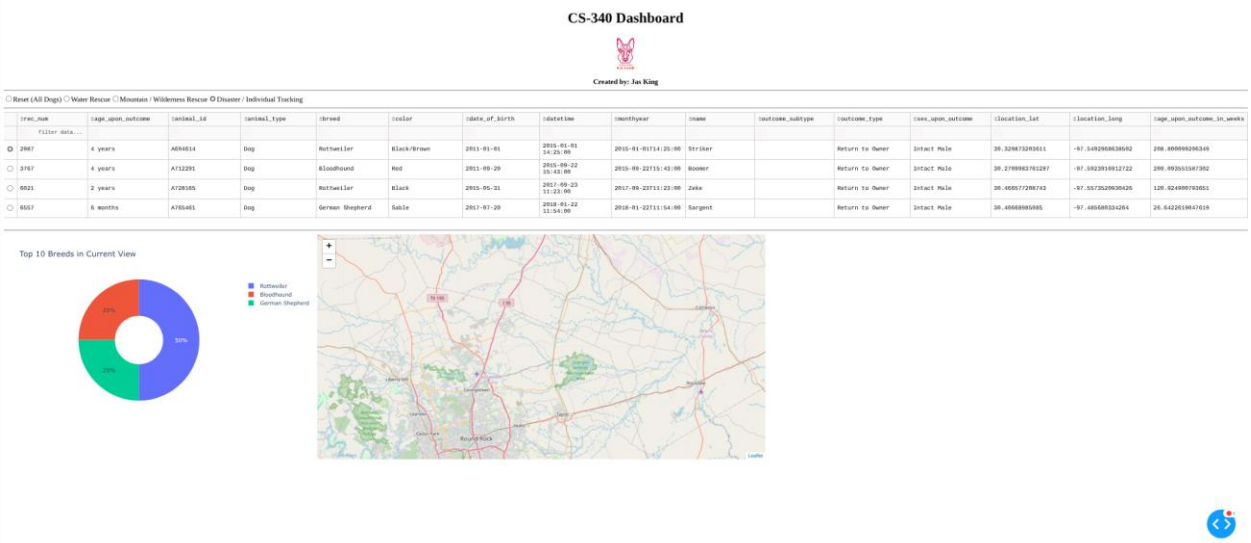
This screenshot shows the dashboard after selecting the Water Rescue filter, with the table, chart, and map updated accordingly.

Mountain or Wilderness Rescue Filter Applied



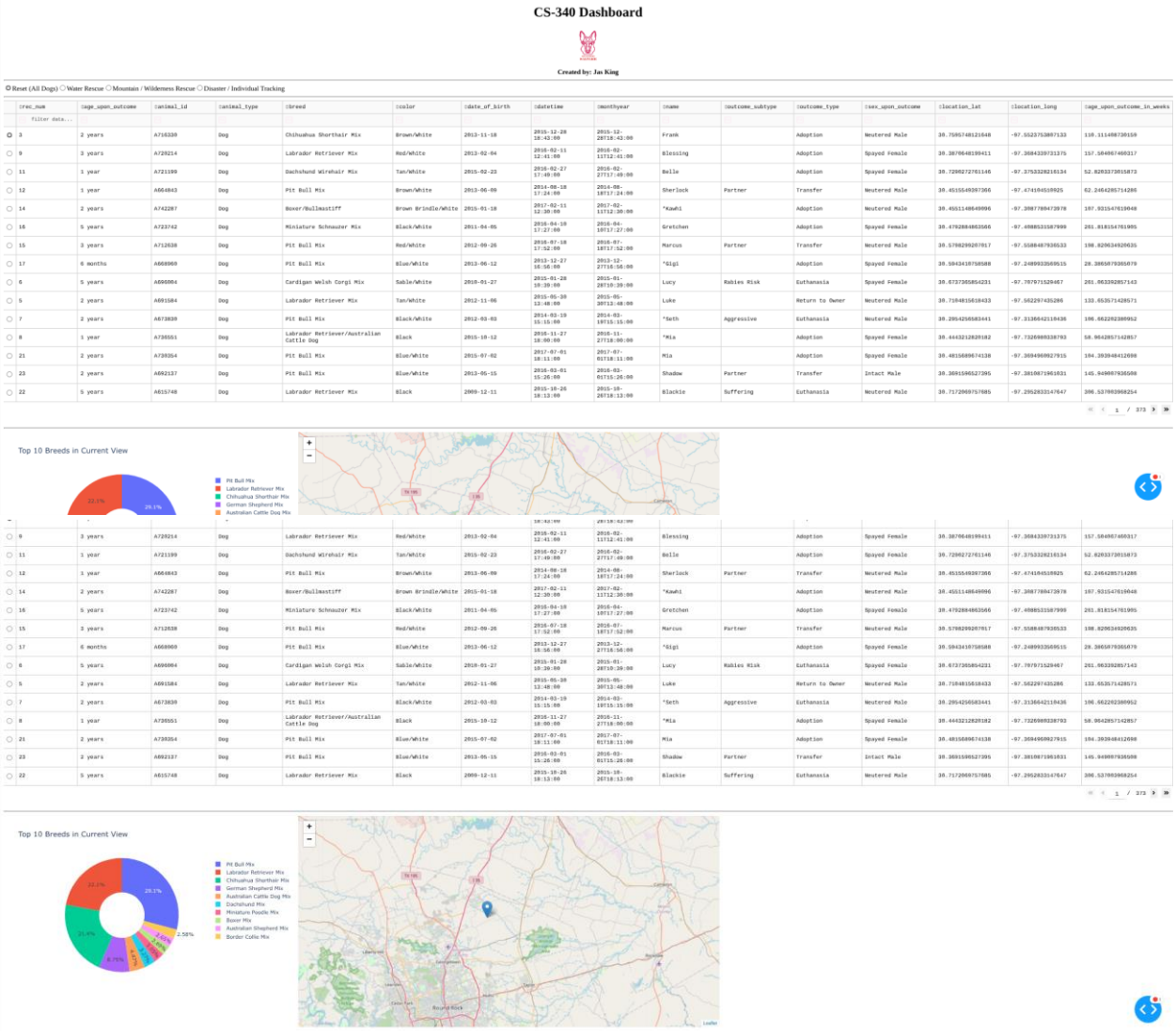
This screenshot shows the dashboard after selecting the Mountain/Wilderness Rescue filter.

Disaster or Individual Tracking Filter Applied



This screenshot shows the dashboard after selecting the Disaster or Individual Tracking filter.

Reset Applied (Unfiltered State Restored)



This screenshot shows the dashboard after resetting the filters to display the full dataset again.

2. Tools Used and Rationale

The following tools and technologies were utilized to implement this project:

- MongoDB
  MongoDB served as the database backend for this project. It is a document-oriented NoSQL database that stores data in flexible, JSON-like documents. This characteristic makes it ideal for managing semi-structured shelter data where records can differ in content. Additionally, MongoDB supports efficient querying, which is essential for filtering animals based on rescue criteria like breed, sex, and age.
- Python and PyMongo
  Python was the main programming language used for this project. The PyMongo library allows Python applications to interact directly with MongoDB. A custom Python CRUD module was created to encapsulate database operations, enhancing the code's modularity, reusability, and maintainability.
- Dash Framework
  The Dash framework was employed to create the web-based dashboard. Dash provides the view and controller components of the application, enabling Python code to define user interface elements and callbacks. Dash callbacks connect user interactions (such as selecting a filter or table row) to database queries and dynamic updates of the table, chart, and map.
- Additional Libraries
  - Pandas was utilized for data manipulation and preparation.
  - Plotly Express was used to create interactive charts.
  - Dash Leaflet was implemented to showcase geolocation data on an interactive map.

3. MongoDB as the Model Component

MongoDB was chosen as the main component of the application because of its flexibility and compatibility with Python. Unlike conventional relational databases, MongoDB does not need a fixed schema, making it simpler to store and manage real-world datasets like animal shelter records.

MongoDB works well with Python via PyMongo, enabling queries to be crafted in Python dictionaries that closely match the format of the stored documents. This feature makes filtering easier and promotes quick development of database-driven applications.

4. Dash Framework as the View and Controller

The Dash framework includes both the view and controller parts of the application. It lets developers create the user interface layout using Python and HTML-like components. User

interactions are managed by callback functions that serve as the controller, updating the dashboard based on user input.

This setup allows the dashboard to stay responsive and interactive without needing additional front-end technologies like JavaScript or HTML files.

5. Resources and Software Used

The following resources and software were used during development:

- MongoDB Community Edition
- PyMongo Python library
- Dash and JupyterDash
- Plotly Express
- Dash Leaflet
- Pandas and NumPy
- Austin Animal Center Outcomes dataset (provided by course materials)

6. Steps Taken to Complete the Project

1. Imported the Austin Animal Center Outcomes dataset into MongoDB with mongoimport.
2. Created a MongoDB user with permissions to read and write for the project database.
3. Developed a Python CRUD module to manage interactions with MongoDB.
4. Constructed the dashboard layout using Dash components.
5. Added interactive filters and callbacks to query the database. Included a data table, geolocation map, and an additional visualization.
6. Tested all states of the dashboard and took screenshots to confirm functionality.

7. Challenges Encountered and Solutions

One of the main challenges faced during development was setting up MongoDB authentication properly so that the Python application could retrieve the data. This problem was fixed by checking user credentials, authentication sources, and database permissions.

Another challenge was making sure that the filter criteria aligned perfectly with the dataset values. This was tackled by examining the dataset and modifying the query logic as needed. Extra attention was given to excluding the MongoDB _id field from the displayed data to avoid problems in the Dash data table.